

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CASE STUDY

Code of Conduct:

I **P.Paneendra (192321072)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

CASE STUDY

[1] A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.

Answer

MD5 is not suitable for storing passwords because it is a fast and outdated hash algorithm. Attackers can use precomputed tables, dictionary attacks, and brute-force techniques to recover weak passwords. MD5 also has serious collision vulnerabilities, although collision resistance is not the main reason password storage with MD5 is unsafe.

A more secure approach is to use a salted, slow password-hashing scheme. If the requirement is specifically to use a SHA-based technique, the company can use PBKDF2-HMAC-SHA-256 with a unique random salt for every password and a sufficiently high iteration count. Simply replacing MD5 with plain SHA-256 is not enough, because SHA-256 is also designed to be fast.

Secure process:

Password → Random Salt + PBKDF2-HMAC-SHA-256 → Stored Password Hash

The database should store the salt, hashing parameters, and derived hash, not the original password.

Conclusion:

MD5 should be completely removed from password storage. Using PBKDF2-HMAC-SHA-256 with a unique salt and high iteration count provides much stronger protection against password-cracking attacks.

[2] A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.

Answer

Unauthorized transactions being approved may indicate weaknesses in the digital signature implementation, key management, or signature verification process.

Possible flaws:

1. The application may not be properly verifying the digital signature before approving transactions.
2. Private keys may be stored insecurely or exposed to attackers.
3. Weak RSA/DSA key sizes or outdated signature algorithms may be used.
4. The system may sign only part of the transaction data, allowing transaction details to be changed.
5. Replay protection may be missing, allowing a valid old signature to be reused.
6. Poor certificate or public-key validation may allow an attacker to impersonate a legitimate user.

Recommended improvements:

- Use modern standards such as RSA-PSS with SHA-256/SHA-384 or ECDSA with SHA-256, with appropriately strong key sizes.
- Hash the complete transaction details, including amount, account number, beneficiary, timestamp, and transaction ID, before signing.
- Protect private keys using secure key storage such as an HSM or equivalent protected key-management system.
- Verify the signature and certificate chain on the server before every transaction approval.
- Include timestamps, nonces, or unique transaction IDs to prevent replay attacks.
- Rotate and revoke compromised keys and certificates.
- Add multi-factor authentication for high-value or unusual transactions.

Conclusion:

The banking system should use a standardized, modern digital-signature scheme with strong key management, complete transaction binding, certificate validation, and replay protection. These measures ensure that only authorized and unmodified transactions are approved.

[3] An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.

Answer

A replay attack in Kerberos occurs when an attacker captures a valid authentication message or ticket and attempts to reuse it later to gain unauthorized access.

How the attack could occur:

1. An attacker captures a valid Kerberos authentication message from the network.
2. The attacker stores the captured ticket or authenticator.
3. If the system does not properly check timestamps or ticket validity, the attacker can resend the captured data.
4. The server may mistakenly accept the replayed authentication as a legitimate request.

Enhancements to prevent replay attacks:

- Use timestamps in Kerberos authenticators and verify that they are within an allowed time window.
- Ensure clock synchronization between clients, servers, and the KDC.
- Use unique session keys for authenticated sessions.
- Implement replay caches so previously used authenticators are rejected.
- Enforce short ticket lifetimes and renew tickets securely when necessary.
- Protect communication channels and use strong encryption.
- Monitor repeated authentication attempts and suspicious ticket reuse.
- Keep the Kerberos KDC and related systems patched and securely configured.

Conclusion:

Kerberos can effectively prevent replay attacks when timestamps, synchronized clocks, replay detection, short-lived tickets, and secure session keys are properly implemented. The enterprise should strengthen these mechanisms and regularly audit its Kerberos configuration.

[4] A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.

Answer

A simple hash function only verifies whether the received message produces the expected hash. It does not provide authentication because the hash does not contain a secret key.

Why the approach fails:

1. The attacker intercepts the original message and its hash.
2. The attacker modifies the message.
3. The attacker calculates a new hash for the modified message.
4. If the receiver accepts the new hash, the modified message appears valid.
5. Therefore, a simple hash cannot distinguish between the legitimate sender and an attacker.

Proposed Solution: HMAC

Use HMAC (Hash-based Message Authentication Code) with a strong hash such as SHA-256.

Process:

Message + Secret Key → HMAC-SHA-256 → Authentication Code

The sender generates an HMAC using the message and a shared secret key. The receiver uses the same secret key to calculate the HMAC again. If both HMAC values match, the message is accepted.

Advantages of HMAC:

- Provides message integrity.
- Provides sender authentication.
- Attackers cannot generate a valid HMAC without the secret key.
- Detects unauthorized message modification.
- HMAC-SHA-256 is suitable for modern applications.

Conclusion

The simple hash approach fails because it provides integrity checking but not authentication. Replacing it with HMAC-SHA-256 using a strong, securely managed secret key prevents attackers from modifying messages without detection.

5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

Answer

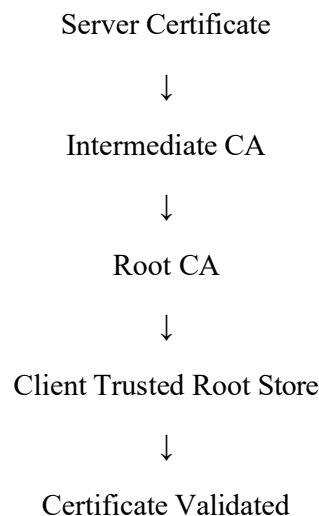
X.509 certificates are used to establish authentication and trust between communicating systems. Frequent trust validation errors usually indicate problems with certificate configuration, expiration, or the certificate trust chain.

Possible issues:

1. The certificate may be expired or not yet valid.
2. The certificate's Common Name (CN) or Subject Alternative Name (SAN) may not match the server domain.
3. The certificate may have been issued by an untrusted or unknown Certificate Authority (CA).
4. Intermediate CA certificates may be missing from the server.
5. The root CA certificate may not exist in the client's trusted certificate store.
6. Certificates may have been revoked.
7. Incorrect system date and time can cause validity checks to fail.
8. Incorrect certificate configuration or an incomplete trust chain can cause validation errors.

Proposed Solutions:

- Use certificates issued by a trusted Certificate Authority.
- Install and configure the complete certificate chain, including required intermediate certificates.
- Regularly monitor certificate expiration dates and renew certificates before they expire.
- Ensure the domain name matches the certificate's SAN.
- Maintain updated and trusted root CA stores on client systems.
- Check certificate revocation using appropriate mechanisms such as CRL or OCSP.
- Synchronize system clocks using reliable time services.
- Automate certificate deployment and renewal where possible.
- Properly protect private keys and restrict access to them.

Trust Chain:**Conclusion:**

The organization should maintain valid certificates, correct CA chains, trusted root certificates, proper domain matching, revocation checking, and accurate system time. These measures ensure reliable X.509 authentication and prevent frequent trust-validation errors.

